Ceremonies for SLIP-39 Group-of-Groups Custody

Organizational Recovery Checklists (DRAFT v0.3)

Dominion R&D, Perry Kundert

August 26, 2026

Contents

1 Executive summary

SLIP-39 splits an encrypted master secret (EMS) across GROUPS, and each group's secret across MEMBER cards. The cryptography answers exactly one question: which subsets of cards reconstruct which secrets. It says nothing about people leaving, cards being photographed, offices burning down, or boards being reorganized. Those are ORGANIZATIONAL failure modes, and surviving them requires ceremonies – scripted procedures in which the math does what math can do, and the checklist does the rest.

This document is written as an aviation-style checklist set. Each failure mode is classified NORMAL / ABNORMAL / EMERGENCY, and each ceremony states its preconditions, its steps, what the mathematics guarantees, what ONLY procedure guarantees, and the executable test that demonstrates it (`test_org_ceremonies.py`). The tests are the ceremony scripts; run them.

Reference scheme used throughout:

```
group_threshold 2 of 3 groups
  G0 "Execs"  2 of 3 member cards
  G1 "Board"  3 of 5 member cards
  G2 "Vault"  2 of 2 member cards
```

2 The custody object model

master secret the wallet seed. Never appears in any ceremony below.

EMS the master secret encrypted (Feistel/PBKDF2) under the passphrase. All splitting and all ceremonies operate on the EMS ciphertext; the passphrase is never requested.

group polynomial degree `group_threshold - 1` polynomial over GF(256) whose point at x=255 is the EMS ciphertext; group i's GROUP SECRET is its point at x=i.

member polynomial per-group polynomial whose x=255 point is that group's secret; each member card is one point on it. A 4-byte HMAC digest of the secret is embedded at x=254, so recovery REFUSES point sets that do not lie on one consistent polynomial.

metadata every card carries (identifier, extendable flag, iteration exponent, group index, group threshold, group count, member index, member threshold). Cards with mismatched common metadata refuse to pool at all.

Two facts drive every ceremony below:

1. A group's secret is a FIXED point of the scheme. Any re-issue of a group must preserve it (that is what keeps new cards combinable with untouched groups), and therefore no group-local ceremony can ever CHANGE it.

2. The member polynomial is FREE. Re-splitting the same group secret onto a fresh random polynomial yields brand-new cards that interoperate with every other group, while the digest check rejects any mixture of old and new cards.

3 What the math guarantees vs what only procedure guarantees

Guarantee	Source
< threshold cards reveal nothing about the secret	math
Mixed old/new cards of a re-issued group are refused (digest)	math
Mismatched metadata refuses to pool	math
Re-issued group still combines with untouched groups	math
Old cards among THEMSELVES keep working after re-issue	math (!)
Old cards cease to exist	PROCEDURE
The group secret in RAM during a ceremony is not exfiltrated	PROCEDURE
The right people were in the room	PROCEDURE

The "(!)" row is the heart of every re-issue ceremony: Shamir sharing has no revocation primitive. A superseded card set remains a valid quorum path until it is physically destroyed. Every checklist therefore ends with DESTROY, and DESTROY is witnessed.

4 Lost vs revealed: regenerate-identical vs re-randomize

The single most important distinction in this document.

LOST (card gone, believed unread) The information still exists in the world at most in one place – nowhere. The correct tool is REGENERATE-IDENTICAL: recover the group's original member polynomial from the surviving cards and re-mint the missing card verbatim. Cheap, group-local, changes nothing. `group_one_mnemonics(cards, expand=N)` is exactly this tool (PR #51's master-quorum `group_ems_mnemonics(pool, expand=...)` is its scheme-wide sibling).

REVEALED (card copied, photographed, or holder untrusted) The information exists in unknown hands. Regenerating identically would be useless (the adversary already has it). The correct tool is RE-RANDOMIZE: recover the group SECRET (not the polynomial), split it onto a FRESH polynomial, and destroy every old card. The adversary's copy is then a point on a dead polynomial: it refuses to combine with the new cards (digest), and its old partners no longer exist. `group_one_mnemonics(cards, member_count=N, revoke=True)` is this tool, and its `report()` carries the destruction requirement.

Using the lost-card tool on a revealed card is a silent security failure: the tests demonstrate that both expand paths reproduce the compromised card verbatim (`test_revealed_card_pr51_expand_is_not_revoca`

5 Ceremony checklists

5.1 C1 – LOST MEMBER CARD (ABNORMAL)

Condition: one G1 card missing; no reason to believe it was read. Surviving members N-1 >= member threshold T.

Preconditions:

- EITHER the group's surviving members (group-local path; nobody else summoned),
- OR a full master quorum of groups (PR #51 `expand` path).

Ceremony:

1. RECOVER + RE-MINT – assemble surviving members on the air-gapped machine; one public call regenerates the ORIGINAL member polynomial and re-mints the missing card verbatim:

   ```
   ceremony = group_one_mnemonics(survivors, expand=5)
   ceremony.minted            # the missing card, byte-identical
   ```

   (Master-quorum alternative: `group_ems_mnemonics(pool, expand=[(1, 5)])`.)

2. VERIFY – re-minted card + T-1 surviving cards recover the group secret; the `ceremony.report()` records that nothing was replaced and nothing revoked.

3. RECORD – log serials/custodians; no destruction required (nothing superseded).

Math guarantees: the re-minted card is byte-identical; nothing else changes. Procedure guarantees: the judgment call that the card was LOST, not REVEALED. If in doubt, run C2 instead – C2 strictly dominates C1 in safety, costing only the re-distribution of the group's cards.

Test: `test_org_ceremonies.py::test_lost_member_card_regeneration`

5.2 C2 – REVEALED / COPIED MEMBER CARD (ABNORMAL) – the key ceremony

Condition: one G1 card suspected copied, photographed, or its holder no longer trusted. Surviving sound members N-1 >= member threshold T (here 4 >= 3).

Preconditions:

- The N-1 sound members of G1, physically assembled.
- Air-gapped machine, wiped storage, no radios.
- A quorum-external witness with the checklist.
- G0 and G2 are NOT summoned and their cards are NOT touched.

Ceremony:

1. RECOVER + RE-SPLIT – the N-1 members enter their cards; one public call recovers G1's GROUP SECRET alone and re-splits it onto a FRESH random member polynomial, ALL metadata pinned (identifier, extendable flag, iteration exponent, group index, group threshold, group count):

   ```
   ceremony = group_one_mnemonics(cards, member_count=5, revoke=True)
   new_cards = ceremony.mnemonics()
   ```

   Neither other groups nor the master secret are involved. Member threshold/count may change in the same call (see C3).

2. VERIFY – on the same machine: (a) any T new cards recover the same group secret; (b) new cards + one untouched group's cards recover the master secret; (c) the compromised card + T-1 new cards is REFUSED (digest).

3. READ THE REPORT – `ceremony.report()` states the destruction requirement: fewer than T ORIGINAL cards may survive anywhere, every leaked or unaccounted-for card counting as surviving. Read it aloud; the witness checks the arithmetic against the ceremony log.

4. DESTROY – burn/shred ALL old G1 cards present, witnessed. The compromised card, wherever it is, has now lost every partner below threshold.

5. DISTRIBUTE – issue new cards; wipe the machine; log the ceremony.

Math guarantees:

- New cards + untouched G0/G2 recover the master secret (group secret preserved).
- Any old+new mixture is refused: digest failure on cross-polynomial sets, or duplicate-index refusal when member indices collide.
- The compromised card alone (or with any sub-threshold set) reveals nothing.

Procedure guarantees:

- Step 4 (DESTROY) is what actually strands the compromised card: mathematically, the old cards among themselves would STILL form a quorum (asserted in the test).
- The group secret exists in RAM during steps 1-2. That is the ceremony's hazard window: the air gap, the wipe, and the witness control it. Note that even full exposure of this one group secret does not expose the master secret (1 < group_threshold) – but it would force escalation to C6.
- No single party sees more than their own cards; the machine, not a person, holds the transient group secret.

Test: `test_org_ceremonies.py::test_revealed_member_card_reissue_ceremony` Also: `test_recover_sing` (step 1's recovery in isolation), `test_group_one_revoke_report` (the report's destruction requirement), `test_digest_rejects_mixed_polynomials` (the refusal mechanism itself).

5.3 C3 – DEPARTED MEMBER (NORMAL)

Condition: a G1 member leaves the organization (planned or unplanned). Treat the departed card as REVEALED regardless of whether it was returned: run C2, optionally resizing the group.

Ceremony: identical to C2, with the resize in the same call – a new member count (e.g. 5 -> 6 to seat a successor plus a spare) and/or a new member threshold (e.g. 3 -> 4):

```
ceremony = group_one_mnemonics(cards, member_threshold=4, member_count=6, revoke=True)
```

Member threshold and count are group-local metadata; changing them does not disturb any other group.

Math guarantees: as C2, plus the new threshold is enforced (3 new cards of a 4-of-6 re-issue are refused).

Test: `test_org_ceremonies.py::test_departed_member_reissue_with_extension`

5.4 C4 – LOST ENTIRE GROUP (ABNORMAL)

Condition: all of G1's cards destroyed or unreachable (site disaster). Remaining groups >= group_threshold (here G0, G2 = 2).

Preconditions: member quorums of group_threshold surviving groups, assembled.

Ceremony:

1. RECOVER – surviving groups each recover their own group secret; interpolate the group polynomial to regenerate G1's group secret (it is forced by the surviving points – this is the same math that recovers the EMS).

2. RE-ISSUE – compose the two public APIs: mint the lost group's single replacement card from the other groups' cards, then reissue that 1-of-1 as a full fresh structure:

   ```
   ((ems, recovered),) = group_ems_mnemonics(pool, expand=[(1, 1)])
   (card,) = recovered[1]
   ceremony = group_one_mnemonics([card], member_threshold=3, member_count=5)
   ```

   The intermediate 1-of-1 card IS the group secret: it is ceremony working material – never distributed, destroyed with the machine wipe. (A bare 1-of-1 replacement group is the stopping point if no multi-card structure is wanted.)

3. VERIFY – new G1 cards + each surviving group recover the master secret.

4. RECORD – there is nothing to destroy but the working material; the old cards are (believed) gone.

WARNING – non-revocation: this ceremony restores AVAILABILITY only. The old group secret cannot change, and the fresh member polynomial only prevents old/new MIXTURES. If the "lost" cards resurface in sufficient number, they still work (asserted in the test). If the loss could plausibly be theft, classify the group secret as compromised and escalate to C6.

This step necessarily reconstructs the EMS-bearing group polynomial: unlike C2, the ceremony momentarily holds master-level material (the EMS ciphertext – still not the master secret, and still no passphrase). Stage it with C6-level controls.

Test: `test_org_ceremonies.py::test_lost_group_replacement`

5.5 C5 – AUTHORITY CHANGE: ADD A GROUP (NORMAL)

Condition: governance adds a custody arm (e.g. a fourth group). Existing groups must keep working.

Reality check: `group_count` is baked into every card's metadata, and cards with different common metadata refuse to pool. A true in-place addition is impossible. The supported ceremony creates a PARALLEL ENCODING of the same EMS:

1. RECOVER – a quorum recovers the EMS (`decode_mnemonics` + `recover_ems`). Ciphertext only; no passphrase; master secret never decrypted.

2. RE-ENCODE – `split_ems` with the enlarged group list (fresh group polynomial, fresh member polynomials, same identifier/flags).

3. VERIFY – new-encoding quorums (including ones using the new group) recover the same ciphertext / master secret; old cards still work among themselves; old and new cards refuse to mix (group_count differs).

4. TRANSITION – either encoding is now a complete custody structure. If the old structure must end, destroy old cards ceremony-wide (witnessed); until then BOTH are live quorum paths, and the org must track both.

Out-of-spec note: mathematically the group polynomial extends past `group_count` – a "phantom" group-3 share can be crafted at the raw level that combines with the ORIGINAL cards (demonstrated in the test). PR #51's `expand` deliberately refuses to mint shares for `group_index >= group_count`, and other SLIP-39 implementations may reject such cards. (`group_one_mnemonics` cannot reach this hazard at all: it only ever mints cards for the one group whose cards it was handed.) Do not build procedure on it; it is documented so that nobody mistakes it for a supported extension mechanism.

Test: `test_org_ceremonies.py::test_add_group_authority_change`

5.6 C6 – COMPROMISED GROUP SECRET (EMERGENCY)

Condition: >= member_threshold of G1's cards copied, or the group secret otherwise assumed exposed. Fewer than group_threshold group secrets exposed in total (so the master secret itself is still information-theoretically safe).

(`group_one_mnemonics` escalates here by refusal, too: `revoke=True` on a threshold-1 group is refused with a pointer to this ceremony, since that group's one card IS its group secret and no re-split can invalidate it.)

Preconditions: member quorums of group_threshold groups; C2-style staging with the strictest controls (this ceremony reconstructs master-level material).

Ceremony:

1. RECOVER – quorum recovers the EMS. Ciphertext only; no passphrase.
2. RE-SHARE – re-encode EVERY group: fresh group polynomial, fresh member polynomials, and (extendable schemes) a NEW identifier so old and new cards are visibly and mechanically distinct.
3. VERIFY – passphrase-free: new-quorum `recover_ems` yields the identical ciphertext. (A single decrypt against a known fingerprint may be added as ground truth, at the cost of touching the master secret.)
4. DESTROY – all old cards of ALL groups, witnessed. Until this completes, the old encoding remains a live quorum path.
5. DISTRIBUTE – new cards to all groups; log.

Math guarantees:

- The exposed G1 group secret is worthless against the new encoding: the new group polynomial is fresh, so new-G1's secret is a different value entirely.
- Old and new cards refuse to mix (identifier differs).
- No on-chain key movement: the master secret never moved, and the adversary's knowledge (one group secret, sub-group-threshold) never sufficed to derive it.

Procedure guarantees: step 4, as always. Old cards among themselves still work until destroyed (asserted in the test).

Extendable-flag note: with `extendable=True` the ciphertext is identifier-independent, so the identifier rotates without the passphrase. With `extendable=False` the identifier is bound into the KDF; rotating it requires decrypting the master secret (passphrase present, plaintext in RAM) and re-encrypting. For organizational custody, create schemes extendable.

ESCALATION – if >= group_threshold group secrets are (or may be) exposed, the master secret must be presumed exposed. No share ceremony helps: move the funds on-chain to a fresh master secret, then stand up a new scheme. This is the only failure mode in this document that leaves the ceremony room.

Test: `test_org_ceremonies.py::test_compromised_group_master_reshare`

6 Revocation summary

Failure mode	Class	Ceremony	Revocation achieved
Lost member card	ABNORMAL	C1	none needed (regenerate-identical)
Revealed member card	ABNORMAL	C2	card stranded: refused with new set, partners destroyed
Departed member	NORMAL	C3	as C2, plus threshold/count change
Lost entire group	ABNORMAL	C4	NONE – availability only; escalate if possibly theft
Add a group	NORMAL	C5	none intended; old encoding retired only by destruction
Compromised group secret	EMERGENCY	C6	group secret rotated; old cards stranded after destroy
>= group_threshold secrets	EMERGENCY	on-chain	not achievable in shares; move funds

In every row, "stranded"/"retired" is the joint product of a math refusal (digest, metadata) and a procedural destruction. Neither alone suffices.

7 API surface

Two top-level ceremony APIs, split by what they must reconstruct:

7.1 `group_ems_mnemonics` – ceremonies that REQUIRE a master quorum

From the PR #51 branch (`feature/group_ems_mnemonics`), unchanged: `group_ems_mnemonics(mnemonics, strict, complete, expand)` pools arbitrary mnemonics, clusters by common metadata, and recovers every reachable EMS while tolerating corrupt, redundant and decoy shares. `expand`[(g, n)]= regenerates group g's ORIGINAL member polynomial within a recovered scheme (n cards), or with n=1 mints a single-card REPLACEMENT for any group – including one with no surviving cards. Owns C4 step 2a, C5 and C6 (with `decode_mnemonics` / `recover_ems` / `split_ems`).

7.2 `group_one_mnemonics` – ceremonies confined to ONE group

`group_one_mnemonics(mnemonics, expand=None, member_threshold=None, member_count=None, revoke=False, strict=False) -> GroupCeremony` collects one group's threshold of member cards – no master quorum, no EMS reconstruction, no other group summoned – and either:

- EXPAND (`expand=N`; N=0 for the default count): regenerate the group's ORIGINAL member polynomial – lost cards re-minted verbatim, additional cards minted exactly as original generation would have; old and new cards freely combine.
- REISSUE (the default): re-split the group secret onto a FRESH member polynomial, all scheme metadata pinned, optionally resizing (`member_threshold` / `member_count`, e.g. 3-of-5 -> 4-of-6). The stance is explicit: AUGMENT (default – old and new sets are parallel encodings, both live) or `revoke=True` (the returned `GroupCeremony.report()` states the destruction requirement that makes the revocation real; refused for threshold-1 groups, with an escalation pointer to C6).

`GroupCeremony` carries the produced cards (`shares` / `mnemonics()`), the cards consumed (`used`), the cards that did not previously exist (`minted`), and the printable ceremony `report()`. Errors are actionable: which group, how many more cards are required, or why the provided cards are inconsistent; pools resolving to more than one group secret are refused as ambiguous.

7.3 Shared internals (refactored so both APIs consume the same pieces)

- `group_common_mnemonics` – configuration-fingerprint clustering.
- `recover_one_rawshares` – the per-grouping combination search (one grouping's cards -> its recoverable group secrets). Deliberately strictness-neutral: silence (an empty result) is the caller's decision – `group_one_mnemonics` raises an actionable error, the recovery pool skips.
- `recover_group_rawshares` – the recovery pool over `recover_one_rawshares` (unchanged pool semantics, now documented as such).
- `locate_ems_rawshares` – level-1 EMS assembly across group combinations.
- `_member_shares` – metadata-pinned card minting (under both `expand_group` and `group_one_mnemonics`).
- `_split_secret` / `_recover_secret_rawshares` – the two polynomial engines: fresh split, and inverse-of-split regeneration.

7.4 Gaps from the first review pass, and their status

1. CLOSED – no public fresh re-split of a single group: `group_one_mnemonics` reissue (C2/C3 are single public calls; the old test-local construction survives only as the regression cross-check).
2. CLOSED – group-local regeneration demanded a master quorum: `group_one_mnemonics(cards, expand=N)` re-mints from the group's own survivors alone.
3. CLOSED – silent refusal: `group_one_mnemonics` raises errors stating the group, the shortfall, or the inconsistency. `recover_group_rawshares` keeps its backward-compatible pool semantics; `recover_one_rawshares` is the strictness-neutral piece both build on.
4. CLOSED (by composition) – multi-card replacement of a LOST group: `group_ems_mnemonics(pool, expand=[(g, 1)])` mints the group-secret-bearing 1-of-1, then `group_one_mnemonics([card], member_threshold`..., member_count=...)= reissues it as a full structure (C4 step 2).
5. Guard rails retained and extended: `group_index >= group_count` refused; expand below a provided member index refused (with the reissue alternative named); multi-card threshold-1 groups refused; revoke of threshold-1 groups refused with escalation.

7.5 Remaining gaps and open design questions

- No CLI surface for `group_one_mnemonics` yet; PR #51 ships `shamir expand`, and a matching `shamir reissue` (or `regroup`) command is the obvious next step.
- The ORIGINAL member count is not recoverable from cards; defaults use `max(2 * threshold, highest index + 1)`. A real ceremony states its counts explicitly.
- Adding a group (C5) and rotating a group secret (C6) remain master-level BY CONSTRUCTION – no group-local API can ever close them; they are `group_ems_mnemonics`'s side of the symmetry.

8 Test map

All tests deterministic (seeded PRNG through the `RANDOM_BYTES` hook); each asserts both required recoveries and required refusals (`MnemonicError`).

Test (test_org_ceremonies.py)	Demonstrates
test_baseline_scheme_recovers	reference scheme quorums and refusals
test_recover_single_group_secret_isolated	C2 step 1's recovery, in isolation
test_revealed_member_card_reissue_ceremony	C2 end-to-end (public API)
test_revealed_card_pr51_expand_is_not_revocation	lost-vs-revealed distinction
test_lost_member_card_regeneration	C1 (all three paths + cross-check)
test_departed_member_reissue_with_extension	C3 (resize 3/5 -> 4/6, public API)
test_lost_group_replacement	C4 (API composition) + non-revocation
test_add_group_authority_change	C5 + phantom-group caveat
test_compromised_group_master_reshare	C6 end-to-end
test_digest_rejects_mixed_polynomials	the refusal mechanism itself
test_group_one_expand_group_local	expand: extension, defaults, refusals
test_group_one_reissue_defaults	the bare-call refresh, pinning
test_group_one_reissue_resize	resizes incl. 1-of-1 collapse
test_group_one_augment_stance	parallel encodings; mixtures refused
test_group_one_revoke_report	destruction requirement; 1-of-N refusal
test_group_one_actionable_errors	the closed silent-refusal gap
test_group_one_matches_private_reissue	public == private construction

Run: `make nix-venv-test` (or `pytest` inside the project venv).

9 Appendix A: Structural notes for further research

Everything below is implied by the implementation, but it took the ceremony work to make it explicit. Read this before extending the ceremony APIs.

9.1 The two-level Shamir structure

A SLIP-39 scheme is the same construction applied twice, byte-wise over GF(256):

LEVEL 1 (master) one polynomial of degree `group_threshold - 1`. Its value at x=255 is the EMS ciphertext, at x=254 the level-1 digest share; group i's GROUP SECRET is its value at x=i. Groups are POINTS on this polynomial.

LEVEL 2 (member; one polynomial per group) degree `member_threshold - 1`, value at x=255 is that group's secret, at x=254 the level-2 digest share; member card j is its value at x=j. Members are POINTS on their group's polynomial.

Consequences that drive every ceremony:

- Any threshold-many points determine the WHOLE polynomial, not just the secret. `_recover_secret_rawsha` inverts `_split_secret`: it regenerates every original point – lost cards, additional cards, even points past the declared count (the "phantom group" caveat in C5). This is why `group_one_mnemonics(cards, expand=N)` needs nothing but one group's own threshold of cards, and why "lost" is never "gone" while a threshold survives.
- A group's secret is a FIXED point of the level-1 polynomial. No group-local operation can change it, and every group-local reissue must preserve it exactly – that is precisely what keeps the new cards combinable with untouched groups, and precisely why a COMPROMISED group secret cannot be fixed group-locally (C6).

- A fresh split of the same secret is genuinely re-randomized for every `member_threshold >= 2`: `_split_secret` draws `threshold - 2` fully random points plus the random part of the digest share. For `member_threshold == 1` there is NO free entropy – the card IS the secret, every reissue reproduces it verbatim, and revocation by re-split is impossible (`group_one_mnemonics` refuses `revoke=True` there and points at C6).

9.2 What the digest verifies, and at which level

Each level embeds `HMAC-SHA256(random_part, shared_secret)[:4]` at x=254 of THAT level. Recovery re-derives it and refuses point sets that do not lie on one consistent polynomial:

- The LEVEL-2 digest is what strands a reissued group's old cards in mixed sets: old and new cards lie on different member polynomials, so any old+new mixture fails the digest (or, on an index collision, the uniqueness check).
- The LEVEL-1 digest is what refuses decoy or incompatible GROUP secrets when combining groups.
- A threshold-1 polynomial has NO digest at either level (nothing to verify; the share is the secret). Refusals are integrity checks, not proofs: a wrong mixture passes with probability 2^-32 per attempt.

9.3 Metadata pinning, and why

Every card carries (identifier, extendable flag, iteration exponent, group index, group threshold, group count, member index, member threshold). Cards whose COMMON parameters (identifier, extendable, iteration exponent, group threshold, group count) differ refuse to pool before any mathematics is attempted – so any ceremony that mints cards meant to interoperate with deployed groups MUST pin all of them (`_member_shares` exists for exactly this). The remaining fields are group-local: member threshold and member count may change in a reissue without disturbing any other group; group index selects which level-1 point the cards serve. Note the metadata is honor-system: it describes the polynomials but is not bound to them cryptographically (see the phantom-group demonstration in C5).

9.4 Regenerate-identical vs re-randomize (same secret, different security)

Both operations preserve the group secret; they differ only in the member polynomial – and that difference is the entire security story:

- REGENERATE-IDENTICAL (`expand`): deterministic interpolation, no entropy consumed, output cards are the original cards. An adversary's copy remains valid. Availability tool only.
- RE-RANDOMIZE (`reissue`): fresh entropy, new polynomial, old and new sets mutually refusing. Confidentiality tool – but only in combination with destruction of the old set (below).

9.5 Augment vs revoke

A reissue's two stances are cryptographically IDENTICAL; the stance is an organizational commitment, which is why the API records it explicitly:

AUGMENT old set and new set are parallel encodings of the group; each independently satisfies it; mixtures are refused by the level-2 digest. Both sets are live quorum paths and both must be tracked.

REVOKE augment's mathematics plus a destruction obligation. The revocation is real only once fewer than `member_threshold` ORIGINAL cards survive anywhere (every leaked or unaccounted-for card counting as surviving). Software cannot destroy paper, so `group_one_mnemonics(..., revoke=True)` returns the requirement in its ceremony `report()`; the checklist executes it. Revocation is always the joint product of a math refusal and a procedural destruction – neither alone suffices.

9.6 RAM-reconstruction hazard windows

Every ceremony momentarily holds secret material in process memory; ceremonies differ in how much. Stage controls (air gap, wiped storage, witness, machine wipe) proportional to the window:

Operation	In RAM during the ceremony
`group_one_mnemonics` (expand or reissue)	ONE group secret (+ its member polynomial)
`group_ems_mnemonics`, `recover_ems` (C4-C6)	level-1 material: group secrets + EMS ciphertext (still encrypted)
`combine_mnemonics` (never in a ceremony)	the MASTER SECRET, decrypted

Exposure of one group secret is survivable ($1 <$ group_threshold; escalate to C6); exposure of level-1 material identifies every group but still needs the passphrase; only the last row is catastrophic. Prefer the smallest window that performs the ceremony – which is the reason `group_one_mnemonics` exists.